

SMC CPU Cluster — Test Plan

- [SMC CPU Cluster — Detailed Test Plan](#)
 - [Contents](#)
 - [1. Objectives](#)
 - [2. Scope \(in / out\)](#)
 - [2.1 In scope](#)
 - [2.2 Out of scope \(covered by other plans\)](#)
 - [3. Verification strategy](#)
 - [3.1 Tier model](#)
 - [3.2 Methods](#)
 - [3.3 Determinism guarantees](#)
 - [4. Environment](#)
 - [5. Test bench architecture](#)
 - [6. Stimulus & checking primitives](#)
 - [7. Feature → test traceability matrix](#)
 - [8. Detailed test cases \(cluster tb \)](#)
 - [8.0 Smoke \(sc main , before cluster tb top \)](#)
 - [8.1 4-hart construction and binding](#)
 - [8.2 Reset-state matrix \(4 harts\)](#)
 - [8.3 Memory routing \(fast-mem + MMIO\)](#)
 - [8.4 smc axi extension on MMIO transactions](#)
 - [8.5 Step\(\) and WFI flag](#)
 - [8.6 IRQ aggregator and WFI wake](#)
 - [8.7 PLIC integration — CPU → PLIC → CPU \(TC-CPU-009 partial\)](#)
 - [8.8 Scratchpad SRAM init handshake \(TC-CPU-004 / TC-CPU-005\)](#)
 - [8.9 CPU-Control register file](#)
 - [9. Coverage plan](#)
 - [9.1 Functional coverage \(instrumentation roadmap\)](#)
 - [9.2 Code coverage targets](#)
 - [9.3 Coverage closure plan](#)
 - [10. Negative-test catalogue](#)
 - [11. Regression workflow](#)
 - [11.1 Local regression](#)
 - [11.2 CI regression](#)
 - [11.3 Coverage regression](#)
 - [11.4 ASan regression](#)
 - [11.5 Triage protocol](#)
 - [12. Pass / fail criteria & exit conditions](#)
 - [13. Future tests & open work](#)
 - [13.1 PLIC integration — status](#)
 - [13.2 Non-PLIC future work](#)
 - [14. Test results — current baseline](#)

SMC CPU Cluster — Detailed Test Plan

Document: 03_CPU_Cluster_Test_Plan.md **Module under test (MUT):** smc::smc_cpu_cluster (cpu_cluster/include/, cpu_cluster/src/) **Reference test bench:** cpu_cluster/test/cluster_tb.cpp (single self-checking binary, plic_tb style) **Status:** Active — cluster_tb passes on SystemC 2.3.4 / GCC 11+ / Linux x86_64 **Companion docs:** - 01_SMC_Architecture.pdf §3 — IP role and modeling parameters - 02_SMC_IP_LowLevel_Design.pdf §3 — TLM interface, IRQ inputs, CPU-Control register file, AXI extension contract - 03_SMC_Test_Plan.pdf §A.1–A.6, §B.3 — project-wide methodology and the TC-CPU-001..010 catalogue this plan refines

Contents

1. [Objectives](#)
 2. [Scope \(in / out\)](#)
 3. [Verification strategy](#)
 4. [Environment](#)
 5. [Test bench architecture](#)
 6. [Stimulus & checking primitives](#)
 7. [Feature → test traceability matrix](#)
 8. [Detailed test cases \(Tier-0 + Tier-1\)](#)
 9. [Coverage plan](#)
 10. [Negative-test catalogue](#)
 11. [Regression workflow](#)
 12. [Pass / fail criteria & exit conditions](#)
 13. [Future tests & open work](#)
 - 13.1 [PLIC integration — status \(done / open\)](#)
 - 13.2 [Non-PLIC future work](#)
 14. [Test results — current baseline](#)
-

Related documents. This test plan refers heavily to 06_CPU_Cluster_Architecture.md — the markdown top-level architecture & implementation reference (chiplet view, internal block view, file layout, class / component / sequence diagrams, build-graph, and §3 conformance summary). Read that first if you want the *what* and the *how*; this document is the *how-we-verify*.

1. Objectives

The CPU Cluster test plan must demonstrate that the SystemC model:

1. **Conforms** to 02_SMC_IP_LowLevel_Design.pdf §3 (CPU Cluster IP) — TLM-2.0 sockets data / mmio / ifetch / ctrl, per-hart IRQ inputs, smc_axi_extension, CPU-Control register file at BASE+ 0x001_0000, and the iss_hart debug API.
2. **Implements correctly** every flow described in §3.5–§3.10 (per-hart step loop with temporal decoupling, bus-bridge routing, IRQ aggregation, CPU-Control register decode, prot[3] AMO-lock propagation, prot[2] privilege bit).
3. **Honours** the TLM-2.0 contract — b_transport initiator usage on the three egress sockets and b_transport target on ctrl.

4. **Realises** the §B.3 TC-CPU-001..010 specification where the wrapper owns the behaviour; peer-IP legs use behavioral stubs in `cluster_tb` (PLIC MMIO + IRQ, scratchpad SRAM init handshake) until production IPs land on the fabric.
 5. **Is integration-ready** — the Whisper ISS backend boots simple hand-coded RV64 image and the wrapper's TLM/IRQ contract is testable by peer IPs without any further changes to `smc_cpu_cluster`.
-

2. Scope (in / out)

2.1 In scope

- Self-checking `cluster_tb` binary (macro checks, `ALL TESTS PASSED`) for every wrapper-owned design point listed in §3.3–§3.10.
- One SystemC elaboration / one `sc_start()` in `cluster_tb_top::run()` (same pattern as `peripherals/plic/test/plic_tb.cpp`).
- Integration phases inside `cluster_tb`:
 - **CPU → PLIC → CPU** (hart firmware programs PLIC; TB only drives `src_in`)
 - **Scratchpad SRAM init handshake** (TC-CPU-004 / TC-CPU-005 via `ScratchpadSramStub`)
- Back-door inspection through `iss_hart::*` debug accessors (`get_pc`, `read_csr`, `is_wfi`, `last_commit`, `current_priv`, `inject_nmi`).
- Coverage instrumentation through `-DSMC_ENABLE_COVERAGE=ON` (`gcov` + `gcovr` → HTML report + console summary).
- §A.5 watchdog: `cluster_tb_top` carries a 120 ms simulated-time watchdog so a stuck WFI fail-stops with `SC_REPORT_FATAL`.

2.2 Out of scope (covered by other plans)

This plan verifies the **cluster wrapper** and its **hooks to peer IPs**. It does not replace full peripheral-IP or SoC-level test plans.

Aspect	Owner / document	Relationship to <code>cluster_tb</code>
Whisper ISA conformance	<code>tenstorrent/whisper</code> upstream	—
RTL co-simulation vs Rocket / Chipyard	Co-simulation (future)	—
Cycle-accurate MMIO / IRQ timing	RTL verification	LT functional only here
<code>axi_filter</code> on cluster egress	<code>axi_filter</code> test plan	—
SMC fabric multi-master arbitration	<code>smc_fabric</code> test plan	—
PLIC IP functional matrix (all sources, thresholds, claim rules)	<code>peripherals/plic</code> + <code>plic_tb</code>	In regression via linked <code>plic.cpp</code> ; not re-tested exhaustively in <code>cluster_tb</code>
Cluster ↔ PLIC integration (CPU MMIO + <code>irq_ext</code> + ISR)	This plan §8.7	In scope — CPU → PLIC → CPU
Production <code>riscv_plic0.c</code> / full SoC IRQ map	SMC firmware / SoC test plan	Open
CLINT IP (mtime, mtimecmp, MSIP MMIO)	<code>peripherals/clint</code> (future)	Not in scope — only <code>irq_sw/irq_timer</code> wire stubs §8.6
Boot-ROM contents / full scratchpad map	Firmware boot-flow test plan	—
Scratchpad on <code>cluster.data</code> @ <code>0xC006_0000</code>	Future fabric router	Init handshake only (<code>ScratchpadSramStub</code>)
Power / DFT / MBIST	RTL DV plan	—

3. Verification strategy

3.1 Tier model

Aligned with `03_SMC_Test_Plan.pdf` §A.6:

Tier	Goal	Where
Smoke	Whisper System constructs	cluster_tb preamble in sc_main
Unit	Per-feature wrapper correctness	cluster_tb phases in run()
Integration	PLIC + scratchpad behavioral peers	cluster_tb PLIC + SRAM phases
System	Boot firmware end-to-end on Whisper	smc_top boot test (future)
Co-simulation	Whisper vs Rocket commit-log diff	tools/whisper_to_rtl_trace.py (future)

3.2 Methods

Method	Purpose
Directed tests	Trigger a specific §3 requirement and check the observable response.
iss_hart back-door assertions	Inspect PC / CSR / WFI / commit state with no perturbation.
Hand-coded RV64 in fast-mem	Drive the step loop without depending on an ELF toolchain.
CapturingRamStub on egress	Byte-accurate reconstruction of multi-byte writes regardless of TLM granularity.
expect_irq polling	Bounded-time rising-edge detection on sc_in<bool> ports.
Watchdog fixture	SC_REPORT_FATAL on hung simulation; budget set per test.
§A.4 coverage gating	gcovr summary checked against the wrapper baseline after every regression.
§A.5 stderr quietness gating	cluster_tb prints only the SystemC banner; warnings escalate to FAIL.

3.3 Determinism guarantees

The test bench is fully deterministic:

- No random stimulus (no std::rand, no time-of-day seeds).
- Single-threaded SystemC scheduler; per-hart SC_THREADs are cooperatively scheduled by the kernel.
- One cluster_tb_top elaboration and one sc_start(); phases run sequentially in run() (no cross-phase module rebuild).
- IRQs are driven by sc_signal<bool> writes followed by wait(SC_ZERO_TIME) settle steps.

A failing test is a hard error — no retries, no flaky regression flags.

4. Environment

Component	Required version
Accellera SystemC	≥ 2.3.4 (built with the same C++ standard: C++20)
Whisper ISS	Pre-built <code>build-Linux/librvcore.a</code> from <code>tenstorrent/whisper</code> with <code>MEM_CALLBACKS=1</code>
Boost	≥ 1.74 (header-only OK; <code>program_options</code> shared lib if Whisper pulls it in)
C++ compiler	C++20 — GCC 11+ or Clang 13+
CMake	≥ 3.20
Accellera CCI	≥ 1.0 when <code>SMC_BUILD_PLIC_INTEGRATION=ON</code> (default)
OS (host)	Linux (primary), macOS (best-effort)
Optional	<code>gcovr</code> for §A.4 coverage; <code>ctest</code> via <code>run_tests.sh --ctest</code>

Reproducible commands: `/home/rmalhotra/.cursor/plans/whisper_cpu_cluster_55f098d3.plan.md`

```
export SYSTEMC_HOME=/localdev/rmalhotra/systemc-2.3.4
export WHISPER_HOME=/localdev/rmalhotra/whisper/whisper
export BOOST_DIR=/localdev/ctr-mharshavardhana/library/boost-1.84.0

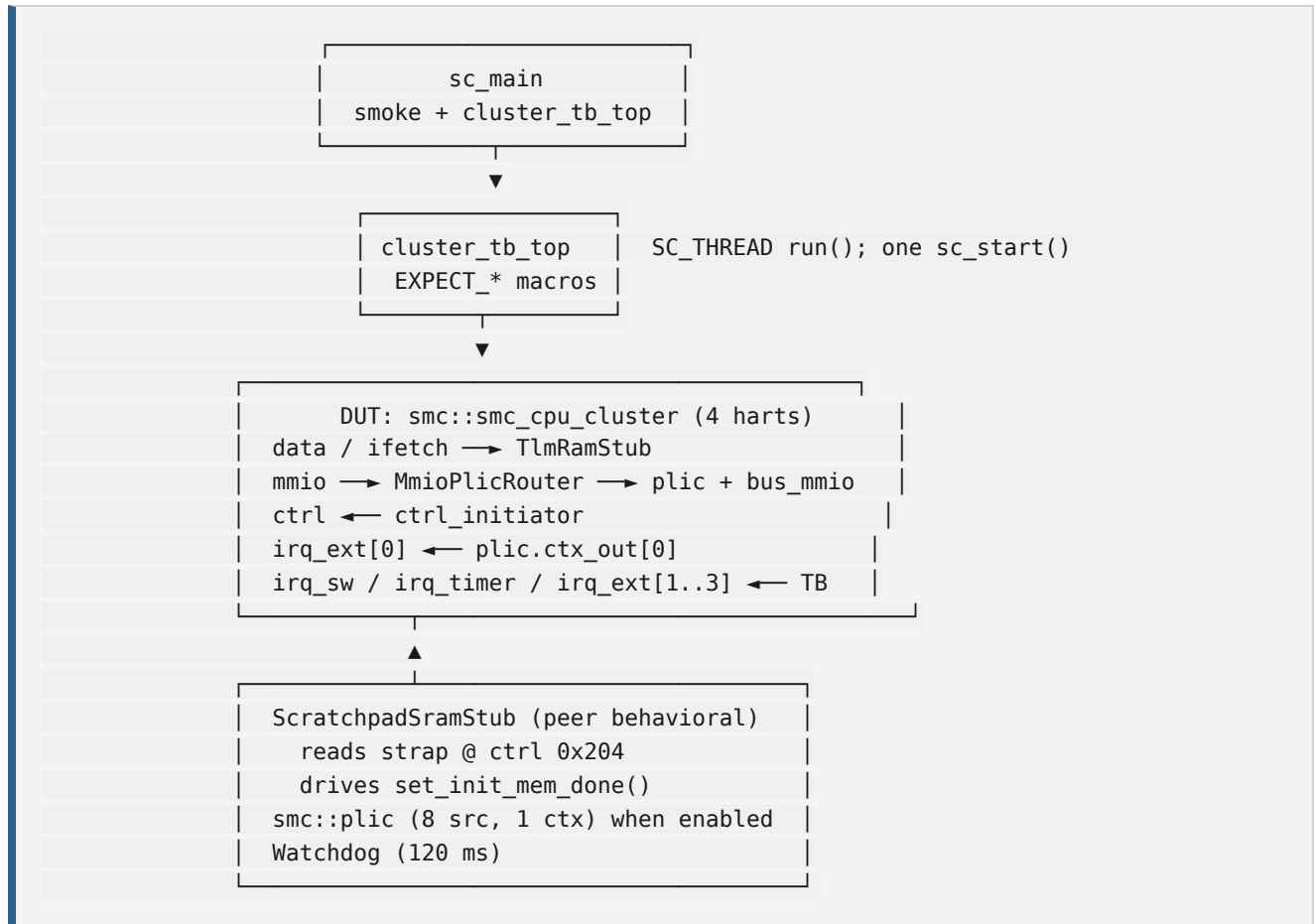
cmake -S cpu_cluster -B cpu_cluster/build -DCMAKE_BUILD_TYPE=Release
cmake --build cpu_cluster/build -j
ctest --test-dir cpu_cluster/build --output-on-failure
```

Or, equivalently, through the wrapper script:

```
cpu_cluster/run_tests.sh           # default: build + run cluster_tb (~8 s)
cpu_cluster/run_tests.sh --clean    # wipe build/ first
cpu_cluster/run_tests.sh --ctest    # run via CTest (same binary)
cpu_cluster/run_tests.sh --coverage # gcov + gcovr → build/coverage/index.html
cpu_cluster/run_tests.sh --asan     # AddressSanitizer
```

`cluster_tb` uses a 120 ms simulated-time watchdog. Requires `CCI_HOME` when `SMC_BUILD_PLIC_INTEGRATION=ON` (default).

5. Test bench architecture



`cluster_tb_top` instantiates:

- `cluster` — `make_default_cluster_cfg(4, reset_pc=0x80); fast-mem [0, 0x10000), MMIO [0x80000000, 0x90000000), RV64GC.`
- `bus_data / bus_ifetch` — `TlmRamStub`; `bus_mmio` — `InspectingRamStub` (captures `smc_axi_extension` on MMIO).
- `MmioPlicRouter` — maps `0x88000000..0x88400000` to PLIC register offsets; other MMIO addresses stay on `bus_mmio`.
- `ScratchpadSramStub` — 4 KiB array; models auto-zero on `sram_rst_n` release and drives `INIT_MEM_DONE` (see §8.9).
- `ctrl` — `smc_test::ctrl_initiator` on the cluster `ctrl` socket.
- `plic_dut` — built when `SMC_BUILD_PLIC_INTEGRATION=ON` (default).

Phases run sequentially inside `run()` using `wait()`; no nested `sc_start()` and no per-phase GoogleTest fixtures.

6. Stimulus & checking primitives

`cluster_tb.cpp` defines local `EXPECT_*` / `ASSERT_*` macros (same semantics as GoogleTest; prints `FAIL file:line` and increments `g_failures`). The §A.3 helpers in `test/include/smc_test_utils.h` provide the SystemC-side primitives:

Primitive	Purpose
<code>smc_test::make_default_cluster_cfg(N)</code>	Returns a populated <code>smc_cpu_cluster::config</code> for N-hart RV64GC with fast-mem <code>[0, 0x10000)</code> .
<code>smc_test::default_buses</code>	Bundles three <code>TlmRamStub</code> instances + a <code>ctrl</code> master; <code>bind(cluster)</code> wires everything in one call.
<code>smc_test::bind_signal_drivers(cluster, ...)</code>	Binds three <code>sc_vector<sc_signal<bool>></code> arrays to <code>irq_sw</code> / <code>irq_timer</code> / <code>irq_ext</code> .
<code>smc_test::TlmRamStub</code>	Minimal TLM target: <code>TLM_OK_RESPONSE</code> for any access; zero-fill on read.
<code>smc_test::CapturingRamStub</code>	Byte-indexed <code>std::map</code> target; reads served from the map; reconstructs N-byte writes from N 1-byte fans.
<code>smc_test::smc_master</code>	<code>read32</code> / <code>write32</code> / <code>read64</code> / <code>write64</code> helpers driving the <code>ctrl</code> target socket from the testbench.
<code>smc_test::expect_irq(port, deadline)</code>	Polls <code>sc_in<bool></code> for a rising edge within <code>deadline</code> ; returns false on timeout.
<code>smc_test::Watchdog(budget)</code>	<code>SC_THREAD</code> that calls <code>SC_REPORT_FATAL</code> after <code>budget</code> sim-time unless <code>cancel()</code> is called first.

Hart back-door API used by tests (`cluster.hart(i).*`):

```

cluster.hart(i).reset();
cluster.hart(i).get_pc();
cluster.hart(i).read_csr(CSR_*);
cluster.hart(i).is_wfi();
cluster.hart(i).last_commit();      // { pc, opcode, trapped, trap_cause, was_wfi, priv }
cluster.hart(i).current_priv();
cluster.hart(i).step(K);            // direct test-side step (not used during SC_THREAD
    runs)
cluster.inject_nmi(i, cause);        // wake parked hart + take NMI

```

Common CSR / opcode constants (declared in `smc_test_utils.h`):

```

CSR_MSTATUS / CSR_MIE / CSR_MTVEC / CSR_MEPC /
CSR_MCAUSE / CSR_MIP / CSR_MISA / CSR_MHARTID

MIP_MSIP = 1<<3, MIP_MTIP = 1<<7, MIP_MEIP = 1<<11

OP_NOP / OP_WFI / OP_J_SELF / OP_ADDI_X1_X0_5 / ...

```

7. Feature → test traceability matrix

Each §3 design point from `02_SMC_IP_LowLevel_Design.pdf` is anchored to at least one Tier-0 / Tier-1 binary. Items still gated on a peer SMC IP are flagged.

#	\$	Design point	Verified by (cluster_tb phase)
F1	3.3	Three initiator sockets <code>data / mmio / ifetch</code>	memory routing, <code>smc_axi_extension</code>
F2	3.3	<code>pick_socket()</code> routing by <code>mmio_lo / mmio_hi</code> window	memory routing
F3	3.3	<code>ctrl</code> target socket, 8 KiB CPU-Control window	CPU-Control register file
F4	3.4	Per-hart <code>iss_hart</code> + Whisper backend; <code>sc_vector</code> wiring	4-hart construction, reset-state
F5	3.5	Per-hart <code>SC_THREAD</code> execution loop with temporal decoupling	<code>step()</code> and WFI flag
F6	3.5	<code>step(K)</code> batched stepping ($K = \text{quantum_insts}$)	<code>step()</code> and WFI flag
F7	3.5	WFI park (<code>wfi_event_[i]</code>) + wake on IRQ aggregator notify	<code>step/WFI</code> , IRQ aggregator
F8	3.6	Fast-mem path for addresses in <code>[fast_mem_lo, fast_mem_hi)</code>	memory routing
F9	3.6	TLM slow path for everything outside fast-mem	memory routing, <code>smc_axi_extension</code>
F10	3.7	<code>irq_sw</code> → MIP[MSIP] (bit 3)	IRQ aggregator
F11	3.7	<code>irq_timer</code> → MIP[MTIP] (bit 7)	IRQ aggregator
F12	3.7	<code>irq_ext</code> → MIP[MEIP] (bit 11)	IRQ aggregator
F13	3.7	Parked hart wakes on rising IRQ edge	IRQ aggregator
F14	3.8	<code>RESET_VECTOR_N[i]</code> write → <code>hart(i).set_reset_pc()</code>	CPU-Control register file
F15	3.8	<code>CORE_ENABLE</code> bit- i clear parks hart on <code>core_enable_event_[i]</code>	CPU-Control register file
F16	3.8	<code>LOCAL_BASE</code> reads back as <code>cfg.local_base_default</code> (RO)	CPU-Control register file
F17	3.8	<code>GLOBAL_BASE / REGION_SIZE</code> round-trip RW	CPU-Control register file (partial)
F18	3.8	<code>INIT_MEM_DONE</code> is RO; peer drives via <code>set_init_mem_done()</code>	SRAM handshake, CPU-Control hook
F19	3.8	<code>DISABLE_SRAM_AUTO_INIT</code> RW strap	SRAM handshake (TC-CPU-005)
F20	3.8	<code>MEM_REPAIR_STATUS</code> is RO; <code>cluster.set_mem_repair_status()</code> sets it	CPU-Control hook (manual, future peer)
F21	3.9	<code>iss_hart::step(K)</code> overload (loops by K)	<code>step()</code> and WFI flag
F22	3.9	<code>iss_hart::inject_nmi(cause)</code> → Whisper <code>setPendingNmi</code> + wake	(not in <code>cluster_tb</code> ; future phase)
F23	3.9	<code>iss_hart::last_commit()</code> populated each <code>step()</code>	<code>smc_axi_extension</code>
F24	3.9	<code>iss_hart::current_priv()</code> returns live <code>privilegeMode()</code>	<code>smc_axi_extension</code>
F25	3.10	<code>smc_axi_extension</code> attached to every outgoing GP	<code>smc_axi_extension</code>
F26	3.10	<code>source_id == cfg.source_id</code> on the extension	<code>smc_axi_extension</code>
F27	3.10	<code>prot[3]</code> set for AMO / LR-SC when <code>amo_lock_detect == true</code>	<code>smc_axi_extension</code>
F28	3.10	<code>prot[2]</code> set when <code>current_priv() != User</code>	<code>smc_axi_extension</code>
F29	A.1	Self-checking runner (no GoogleTest)	<code>cluster_tb</code> + <code>sc_main</code> smoke
F30	A.3	Common helpers in <code>smc_test_utils.h</code>	<code>cluster_tb</code>
F31	A.4	Coverage instrumentation via <code>-DENABLE_COVERAGE=ON</code>	<code>coverage</code> custom target
F32	A.5	Zero SystemC warnings; Watchdog	<code>cluster_tb</code> (120 ms)
F33	A.6	Regression via <code>run_tests.sh</code> / optional CTest	<code>run_tests.sh</code> , <code>test/CMakeLists.txt</code>

#	§	Design point	Verified by (<code>cluster_tb</code> phase)
F34	3.7	PLIC <code>ctx_out</code> → <code>irq_ext</code> → MIP[MEIP] → trap	PLIC CPU → PLIC → CPU
F35	3.7	CPU MMIO programs PLIC; ISR claim/complete	PLIC CPU → PLIC → CPU
F36	3.10	Unified <code>smc_axi_extension</code> ODR-safe across cluster + PLIC TUs	PLIC integration build (link)
F37	3.8	TC-CPU-004: SRAM auto-init → <code>INIT_MEM_DONE</code> asserts	scratchpad SRAM handshake
F38	3.8	TC-CPU-005: disable strap → <code>INIT_MEM_DONE</code> never asserts	scratchpad SRAM handshake

Items F1–F28 cover the wrapper's §3 contract. F34–F36 cover **CPU** → **PLIC** → **CPU** (MMIO configure + `src_in` + firmware ISR). F37–F38 cover the scratchpad init handshake via `ScratchpadSramStub`. Remaining peer-IP items (S-mode PLIC context, BEU NMI, Debug Module, Boot ROM on fabric) are in §13.

8. Detailed test cases (`cluster_tb`)

Single binary `build/test/cluster_tb`. `sc_main` runs a Whisper smoke check, then `cluster_tb_top::run()` executes phases sequentially inside one `sc_start()`. Format: *§ref* → *Setup* → *Stimulus* → *Expected* → *Notes*.

8.0 Smoke (`sc_main`, before `cluster_tb_top`)

- **§ref**: toolchain / Whisper link
- **Setup**: construct `WdRiscv::System<uint64_t>(1, 1, 1, 1 MiB, 4 KiB)`.
- **Expected**: `hartCount() == 1`, `ithHart(0) != nullptr`.
- **Notes**: Same role as legacy `smoke_test`; not a separate binary.

8.1 4-hart construction and binding

- **§ref**: F4 (TC-CPU-001 / 002 wrapper share)
- **Setup**: 4-hart cluster; `CORE_ENABLE = 0x1` (hart 0 only for later phases).
- **Expected**: `num_harts() == 4`, `whisper_system().hartCount() == 4`, each hart `MHARTID == i`, `is_wfi() == false` at start.

8.2 Reset-state matrix (4 harts)

- **§ref**: F4, F14
- **Stimulus**: `hart(i).reset()` for $i \in \{0..3\}$.
- **Expected** (per hart): `get_pc() == RESET_PC (0x80)`, `MHARTID == i`, `MISA != 0`, `is_wfi() == false`.

8.3 Memory routing (fast-mem + MMIO)

- **§ref**: F1, F2, F8, F9 (TC-CPU-003 partial)
- **Setup**: hart 0; `smc_master` back-door; `InspectingRamStub` on MMIO.
- **Stimulus**:
 - Loads/stores in `[fast_mem_lo, fast_mem_hi)` and in `[mmio_lo, mmio_hi)`.
- **Expected**:
 - Fast-mem via ISS buffer (no egress trace on `bus_data` for in-range).
 - MMIO writes captured in `bus_mmio.mem`.

- Round-trip 8/16/32/64-bit MMIO and seeded readbacks.

8.4 smc_axi_extension on MMIO transactions

- **\$ref:** F23–F28 (§3.10)
- **Stimulus:** MMIO write; AMO lock follow-on; privilege change.
- **Expected:** `source_id`, `prot[3]` lock bit, `prot[2]` for U-mode.

8.5 Step() and WFI flag

- **\$ref:** F5–F7, F21
- **Stimulus:** hand-coded sequence ending in `wfi`; `poke_mip(MTIP)`.
- **Expected:** PC progress, `is_wfi()` after WFI, clears on IRQ poke.

8.6 IRQ aggregator and WFI wake

- **\$ref:** F10–F13 (TC-CPU-007 / 008 partial)
- **Setup:** `CORE_ENABLE = 0xF`; all harts in WFI loop.
- **Stimulus:** drive `irq_timer[0]`, `irq_sw[1]`, `irq_ext[2]`.
- **Expected:** correct `MIP` bit per hart; WFI clears; hart 2 PC moves on MEIP.
- **Notes:** `irq_aggregator` must snapshot `is_wfi()` before `poke_mip()`.

8.7 PLIC integration — CPU → PLIC → CPU (TC-CPU-009 partial)

- **\$ref:** F34–F36
- **Build:** `SMC_BUILD_PLIC_INTEGRATION=ON` (default); `CCI_HOME` required.
- **Responsibility split:**
 - **CPU (firmware @ 0x80 / trap @ 0x200):** MMIO programs priority, enable, threshold; sets `mtvec`, `mie.MEIE`, `mstatus.MIE`; `wfi`; ISR does 32-bit claim + complete; sets `PLIC_FLAG_DONE` at `0x0`.
 - **TB only:** `src_sigs[0].write(true/false)` (device IRQ); polls `PLIC_FLAG_DONE` and `PLIC_dbg_pending / ctx_out`.
- **Topology:** `cluster.mmio` → `MmioPlicRouter` (`0x88000000` window) → `plic.reg_socket`; `plic.ctx_out[0]` → `cluster.irq_ext[0]`.
- **Expected:**
 - Hart reaches WFI with `dbg_priority(1)==7`, `dbg_enable(0,1)`.
 - After `src_in[0]`: `PLIC_FLAG_DONE==1`, pending clears after deassert.
- **Notes:** Full PLIC arbitration matrix remains in `peripherals/plic/test/plic_tb.cpp`.

8.8 Scratchpad SRAM init handshake (TC-CPU-004 / TC-CPU-005)

- **\$ref:** F37–F38
- **Model:** `ScratchpadSramStub` — 4 KiB behavioral array; samples `cluster.disable_sram_autoinit()` (ctrl `0x204`) on `sram_rst_n` release.
- **TC-CPU-004** (`DISABLE_SRAM_AUTO_INIT = 0`):
 1. Pulse `sram_rst_n` low → high.
 2. Stub zeros memory, then `cluster.set_init_mem_done(true)`.
 3. **Expected:** `ctrl.read32(0x200) == 1`; `peek(0)==peek(128)==0`.
- **TC-CPU-005** (`DISABLE_SRAM_AUTO_INIT = 1`):
 1. `scratchpad.seed(0xA5)`; pulse reset.
 2. **Expected:** `ctrl.read32(0x200) == 0` after 200 μs; memory still `0xA5`.

- **Notes:** Stub does not yet bind a TLM port on `cluster.data` at `0xC006_0000` (init-only peer). Production `scratchpad_sram` will replace the stub.

8.9 CPU-Control register file

- **\$ref:** F3, F14–F17, F18 hook, F20
- **Stimulus:** `RESET_VECTOR_N`, `CORE_ENABLE` park/resume, `LOCAL_BASE` RO, manual `set_init_mem_done` / `set_mem_repair_status` hook checks.
- **Expected:** access policies per §3.8; strap `0x204` retained after SRAM phase.
- **Notes:** TC-CPU-004/005 are **not** only register round-trips — see §8.8.

9. Coverage plan

9.1 Functional coverage (instrumentation roadmap)

The current suite is directed. Coverage instrumentation hooks expected in follow-up commits:

Bin / cross	Implementation hook
Per-socket initiator usage (<code>data</code> / <code>mmio</code> / <code>ifetch</code>)	Counter inside <code>tlm_access()</code> after <code>pick_socket()</code>
Per-egress access size distribution (1/2/4/8 B)	Counter in <code>tlm_access()</code> before <code>b_transport</code>
Per-hart <code>irq_sw</code> / <code>irq_timer</code> / <code>irq_ext</code> rising edges	Counter inside <code>irq_aggregator(i)</code>
Per-hart WFI park / wake-up count	Counter inside <code>hart_thread(i)</code> WFI branch
<code>CORE_ENABLE</code> bit- <code>i</code> toggles	Counter inside <code>apply_core_enable()</code>
<code>RESET_VECTOR_N[i]</code> writes	Counter inside <code>ctrl_b_transport()</code> decode
<code>smc_axi_extension prot[3]</code> set / clear	Counter inside <code>tlm_access()</code> extension-population branch
<code>smc_axi_extension prot[2]</code> set / clear	Same
Cross: (<code>priv</code> $\in$ {U,S,M}) $\times$ (<code>cmd</code> $\in$ {R,W})	Sample in <code>tlm_access()</code>

9.2 Code coverage targets

When integrated with `gcov` / `llvm-cov`, the targets are:

Metric	Target	Current baseline (<code>libsmc/cpu/</code>)
Line coverage	$\geq 95\%$	97.1 % (<code>src/</code> , <code>./run_tests.sh --coverage</code>)
Function cov.	$\geq 95\%$	95.5 %
Branch cov.	$\geq 90\%$	88.8 %

The lift to the §A.4 targets is gated on the §13 follow-up items — mostly negative-path coverage in `ctrl_b_transport()` and `tlm_access()`'s error responses.

Untestable lines (e.g. `SC_REPORT_FATAL` paths in the constructor) are explicitly excluded with `// LCOV_EXCL_LINE` comments.

9.3 Coverage closure plan

A test is considered insufficient if any §3 design point in §7 is covered by zero tests, or any covergroup bin in §9.1 has count 0 after the regression. Closure is reached when:

- All feature rows (F1..F38) have at least one passing test.
- All covergroup bins are hit at least once.
- Negative-test catalogue (§10) is fully exercised.
- Code coverage targets in §9.2 are met.
- `cov_build/coverage/index.html` is published as a CI artifact.

10. Negative-test catalogue

#	Negative scenario	Expected response	Covered by
N1	<code>ctrl</code> read with <code>length != 4</code> and <code>length != 8</code>	<code>TLM_BURST_ERROR_RESPONSE</code> (or fatal)	Future (planned)
N2	<code>ctrl</code> access outside the 8 KiB window (after modulo)	Handled by modulo; non-existent regs <code>RAZ/WI</code>	Future (planned)
N3	Write to <code>LOCAL_BASE</code> / <code>INIT_MEM_DONE</code> / <code>MEM_REPAIR_STATUS</code> (RO)	Silently dropped (no state change)	CPU-Control phase (partial)
N4	ELF not found in <code>cluster.load_elf({...})</code>	Returns <code>false</code> ; no SystemC report	Future (planned)
N5	<code>cfg.num_harts == 0</code> or <code>> 4</code>	<code>SC_REPORT_FATAL</code> at elaboration	Future (planned)
N6	<code>cfg.mmio_lo == cfg.mmio_hi</code> (MMIO disabled)	All non-fast-mem traffic egresses on <code>data</code>	Future (planned)
N7	<code>cfg.fast_mem_lo >= cfg.fast_mem_hi</code>	Fast-mem disabled; everything goes TLM	Future (planned)
N8	AMO at a non-fast-mem address	Whisper raises <code>STORE_ACC_FAULT</code> (cause 7)	<code>smc_axi_extension</code> phase (documented)
N9	NMI while hart is parked on <code>wfi_event_</code>	Hart wakes; next <code>step()</code> lands on <code>nmiPc_</code>	Future <code>cluster_tb</code> phase
N10	NMI while hart is parked on <code>core_enable_event_</code>	Hart stays parked; NMI pending until enable	Future (planned)
N11	Concurrent IRQ + NMI	NMI wins on next <code>step()</code>	Future (planned)
N12	<code>irq_aggregator</code> runs before <code>poke_mip()</code> settles (legacy ordering bug)	<code>is_wfi()</code> snapshot must precede the poke	IRQ aggregator phase (regression-pinned)

Items marked `Future (planned)` are tracked in §13.

11. Regression workflow

11.1 Local regression

```
cpu_cluster/run_tests.sh           # build + run cluster_tb (default)
cpu_cluster/run_tests.sh --ctest   # same via CTest
cpu_cluster/run_tests.sh --clean   # wipe build/ first
```

The script:

1. Probes / asserts `SYSTEMC_HOME`, `WHISPER_HOME`, `BOOST_DIR`, `CCI_HOME` (PLIC).
2. Configures CMake (incremental on subsequent runs).
3. Builds `cluster_tb` with `-j<ncpus>`.
4. Runs `build/test/cluster_tb` (or `ctest` with `--ctest`).
5. Exits non-zero if `g_failures != 0` or build fails.

11.2 CI regression

Gate on every commit: `cluster_tb` must print `ALL TESTS PASSED`. The 120 ms watchdog fail-stops wedged simulations.

11.3 Coverage regression

```
cpu_cluster/run_tests.sh --coverage
# → cov_build/coverage/index.html
# → console gcovr summary printed at the end
```

The CI may compare the new gcovr percentages against the §9.2 baseline and fail if any metric regresses by more than 0.5 absolute percentage points.

11.4 ASan regression

```
cpu_cluster/run_tests.sh --asan
```

Uses an isolated `build_asan/` so ASan flags never clobber the Release cache. On Linux, also enables LeakSanitizer (`detect_leaks=1`).

11.5 Triage protocol

On failure:

1. Read the first `FAIL file:line` line from `cluster_tb` stdout.
2. Optionally re-run with `--clean` to rule out a stale build.
3. Add `cluster.hart(i).get_pc() / read_csr(...)` / `last_commit()` adjacent to the failing assertion to capture full hart state.
4. If the failure reproduces under one test case but not in isolation, suspect SystemC kernel state leakage — but every binary is its own process, so this is unlikely; check the build first.

5. If the failure involves the IRQ path, dump `is_wfi()` **before** and **after** every `poke_mip()` — the §3.7 ordering bug is the single most common regression in this area.

12. Pass / fail criteria & exit conditions

Outcome	Criterion
PASS	<code>cluster_tb</code> exits 0 and prints <code>ALL TESTS PASSED</code> .
FAIL	Any <code>EXPECT_*</code> / <code>ASSERT_*</code> fails, or any binary exits non-zero, or <code>SC_REPORT_FATAL</code> fires.
BUILD FAIL	CMake configuration or compilation error — counts as FAIL for CI.
TIMEOUT (CI)	CTest <code>TIMEOUT 60</code> exceeded — counts as FAIL (current actual: < 1 s per binary).
STDERR FAIL (§A.5)	Any binary emits more than the 4-line SystemC banner — counts as FAIL.
ASAN FAIL	<code>--asan</code> mode reports any <code>ERROR: AddressSanitizer</code> — counts as FAIL.
COVERAGE REGRESSION	Any §9.2 metric drops by > 0.5 absolute pp from the published baseline — counts as FAIL (CI policy).

A green CI run requires `cluster_tb` to pass on every supported (SystemC version × compiler × OS) matrix entry.

13. Future tests & open work

13.1 PLIC integration — status

Cluster ↔ PLIC **CPU** → **PLIC** → **CPU** is regression-pinned in `cluster_tb` §8.7. The S-mode context leg and multi-hart PLIC topology remain open.

Done (in the regression today)

Item	Evidence
Build glue: <code>SMC_BUILD_PLIC_INTEGRATION=ON</code> + <code>smc_plic_ip</code>	<code>CMakeLists.txt</code>
CCI discovery + unified <code>smc_axi_extension</code> ODR shim	<code>cpu_cluster/include/smc_axi_extension.h</code> , <code>smc_tlm_extensions.h</code>
<code>MmioPlicRouter</code> at <code>0x88000000</code> → PLIC offsets	<code>cluster_tb.cpp</code>
CPU MMIO programs PLIC (priority / enable / threshold)	<code>load_plic_cpu_firmware()</code> , <code>dbg_priority</code> / <code>dbg_enable</code> readback
CPU ISR claim (32-bit <code>lw</code>) + complete + <code>PLIC_FLAG_DONE</code>	<code>cluster_tb</code> PLIC phase
TB only drives <code>src_in[0]</code>	<code>src_sigs[0]</code>
<code>plic.ctx_out[0]</code> → <code>irq_ext[0]</code> → MEIP → trap @ <code>0x200</code>	<code>cluster_tb</code> PLIC phase
Stand-alone PLIC matrix	<code>peripherals/plic/test/plic_tb</code>

Open / not-yet-done

Item	Priority	Notes
S-mode leg: <code>plic.ctx_out[2*h+1] → irq_ext_s[h] → MIP[SEIP]</code>	High	New <code>irq_ext_s</code> port; <code>num_contexts = 2*num_harts</code>
Multi-hart × multi-context PLIC (4×8)	Medium	Today PLIC phase uses <code>CORE_ENABLE=0x1</code> , one context
<code>riscv_plic0.c</code> / production firmware driver	High	Re-use same topology; ELF instead of hand-encoded insns
PLIC threshold / multi-source arbitration in <code>cluster_tb</code>	Low	Covered by <code>plic_tb</code> ; optional extra phase
<code>smc_top</code> parent owning cluster + PLIC + CLINT	Medium	Reduces per-test boilerplate
Negative MMIO to PLIC (misaligned, OOR) from hart	Low	Extend <code>MmioPlicRouter</code> / PLIC error responses

13.2 Non-PLIC future work

Item	Priority	Notes
Scratchpad TLM on <code>cluster.data @ 0xC006_0000</code> (full map / DMI)	Medium	<code>ScratchpadSramStub</code> is init-only today
Production <code>scratchpad_sram</code> IP replacing behavioral stub	Medium	32 banks, repair/MBIST hooks
TC-CPU-006: NMI from BEU — drive <code>beu_nmi_in[i]</code> once port exists	High	Blocked on BEU IP
TC-CPU-010: Boot ROM execution end-to-end	High	Blocked on Boot ROM IP + scripted boot artefact
<code>inject_nmi</code> phase in <code>cluster_tb</code>	Low	Was in legacy <code>cluster_step_test</code>
<code>ctrl</code> negative paths (<code>length != 4/8</code> , bad command) → TLM error response	Medium	Add <code>raw_xfer</code> helper to <code>smc_master</code>
<code>cfg</code> sanity checks (<code>num_harts</code> / <code>fast-mem</code> / <code>mmio</code> / <code>quantum</code>) at elaboration	Medium	Use <code>sc_report_handler::set_actions</code> to capture
Concurrent NMI + IRQ ordering matrix	Medium	Extend <code>cluster_step_test</code> with a phase 9
Per-egress access-size cover bin (<code>tlm_access</code>)	Medium	Coverage plumbing
Random-stimulus fuzzer driving <code>ctrl</code> register accesses	Low	Bound-checked seeded random; compare reset-state replay
Lockstep co-sim vs Rocket via <code>tools/whisper_to_rtl_trace.py</code>	High	Blocked on the trace adapter (Spike-removal task #7)
4-hart MMIO stress with <code>CapturingRamStub</code> shared across egresses	Low	Useful regression for <code>current_hart_quantum</code> context
<code>debug_irq_in</code> + <code>mhartid</code> -indexed Debug Module hooks	—	Blocked on Debug Module IP

14. Test results — current baseline

Phase (pass string)	§B.3 / feature	Status
smoke: Whisper System constructs	toolchain	PASS
4-hart construction and binding	TC-CPU-001/002	PASS
reset-state matrix (4 harts)	TC-CPU-001/002	PASS
memory routing (fast-mem + MMIO)	TC-CPU-003 partial	PASS
smc_axi_extension on MMIO transactions	§3.10	PASS
step() and WFI flag	—	PASS
IRQ aggregator and WFI wake	TC-CPU-007/008 partial	PASS
CPU configures PLIC, handles MEIP	TC-CPU-009 partial	PASS
scratchpad SRAM INIT_MEM_DONE handshake	TC-CPU-004/005	PASS
CPU-Control register file	TC-CPU-001/002/003	PASS

Result: cluster_tb — ALL TESTS PASSED (0 failures) **Environment:** SystemC 2.3.4 Accellera + CCI 1.0 (PLIC), GCC 11+ (C++20), Linux x86_64 **Wall-clock:** ~8 s (./run_tests.sh --clean) **Coverage:** 97.1 % line coverage on src/ (./run_tests.sh --coverage) **Build flags:** SMC_BUILD_PLIC_INTEGRATION=ON (default), CCI_HOME required for PLIC phase

End of document.